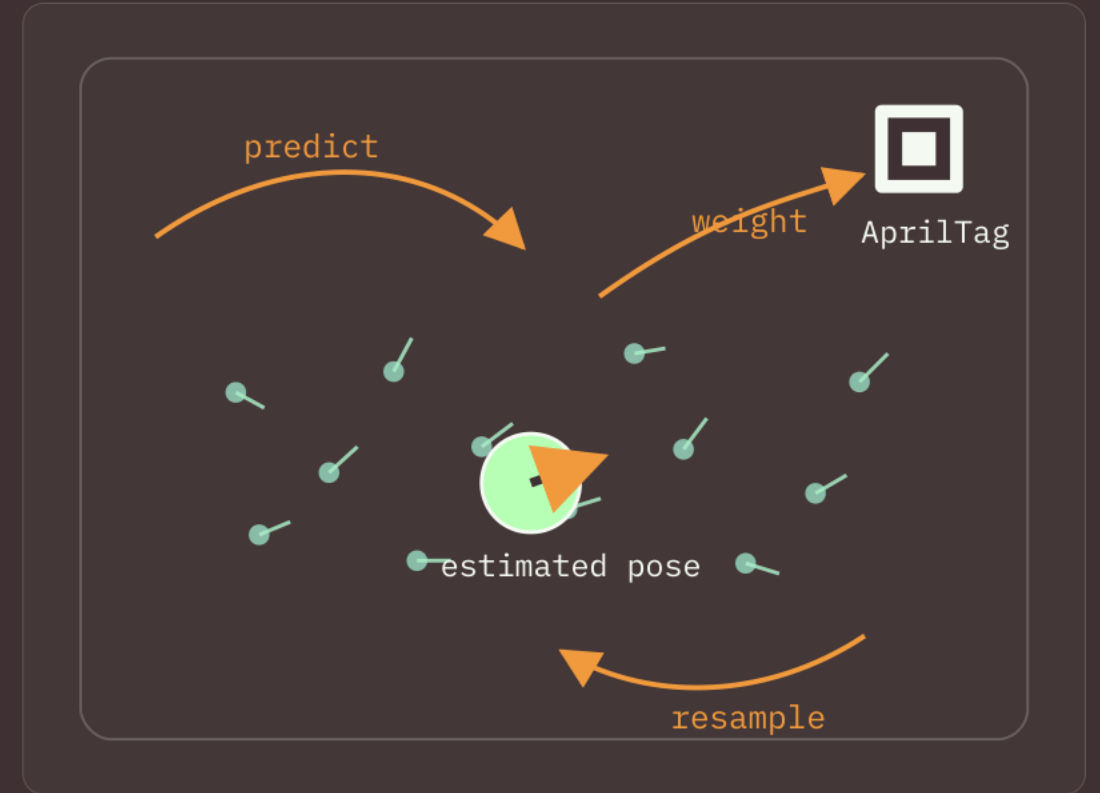


FINAL SECTION · PHYSICAL ROBOT IMPLEMENTATION

Real Duckiebot Particle Filter

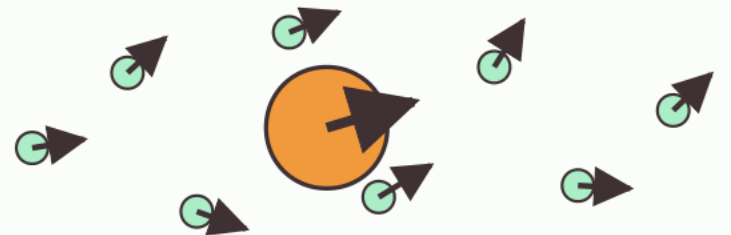
From odometry prediction to
measurement likelihood and adaptive
resampling



A Hypothesis About the Robot Pose

$$p_i = (x_i, y_i, \theta_i)$$

1. Keep many possible robot poses at the same time.
2. Initialize them uniformly across the map region, with random headings.
3. Move every hypothesis when wheel motion arrives.
4. When tags appear, test every particle against the camera observations.
5. Assign larger weights to explanations that fit the observed geometry.
6. Suppress hypotheses that poorly explain the visual evidence.



competing pose hypotheses

A particle is not a sensor reading. It is one competing hypothesis about the robot's current pose.

Odometry Moves Every Hypothesis

$$(x_i, y_i, \theta_i) \rightarrow (x'_i, y'_i, \theta'_i)$$

- Convert left and right encoder tick changes into wheel displacements dL and dR .
- Apply the differential-drive odometry update to every particle.
- Add independent Gaussian noise to each wheel displacement before moving each particle.
- Do not touch the particle weights during odometry-only prediction.

Motion noise keeps the cloud expressive: particles follow the same encoder evidence, but not an unrealistically identical trajectory.

```
for each particle:  
    noisy_dL = dL + wheel_noise()  
    noisy_dR = dR + wheel_noise()  
    particle = differential_drive_update(  
        particle, noisy_dL, noisy_dR  
    )
```

Processing loop with new encoder deltas



Move every particle



No detected tag? Keep predicting with odometry only.

What Is Actually Compared?

$$z = (d, \beta, \psi)$$

- **d**: range from camera to tag on the robot movement plane.
- **x**: tag bearing in the robot frame, computed from projected local x and y.
- **T**: observed tag-face normal yaw after rotating the detected marker normal into the robot frame and projecting it onto the ground plane.

$$\hat{z}_i = (d_i^{\wedge}, \beta_i^{\wedge}, \psi_i^{\wedge})$$

For a candidate particle, the code predicts from a forward camera position offset. Its expected yaw is $T_i^{\wedge} = \text{wrap}(\text{tag } D - \text{particle } D)$.

known tag pose
(tag x, tag y, tag D)

particle pose
(x_i, y_i, D_i)

predict expected observed
($d_i^{\wedge}, \beta_i^{\wedge}, \psi_i^{\wedge}$)

$$e_{d,i} = d - d_i^{\wedge}$$

$$e_{\beta,i} = \text{wrap}(\beta - \beta_i^{\wedge})$$

$$e_{\psi,i} = \text{wrap}(\psi - \psi_i^{\wedge})$$

Angle wrapping prevents directions near +L/and -L/ from being treated as far apart.

Does the Particle Predict the Correct Range?

$$L_{d,i} = \exp\left(-\frac{1}{2} \left(e_{d,i} / \sigma_d \right)^2 \right)$$

small distance
error



likelihood near
1



particle stays
competitive

large distance
error



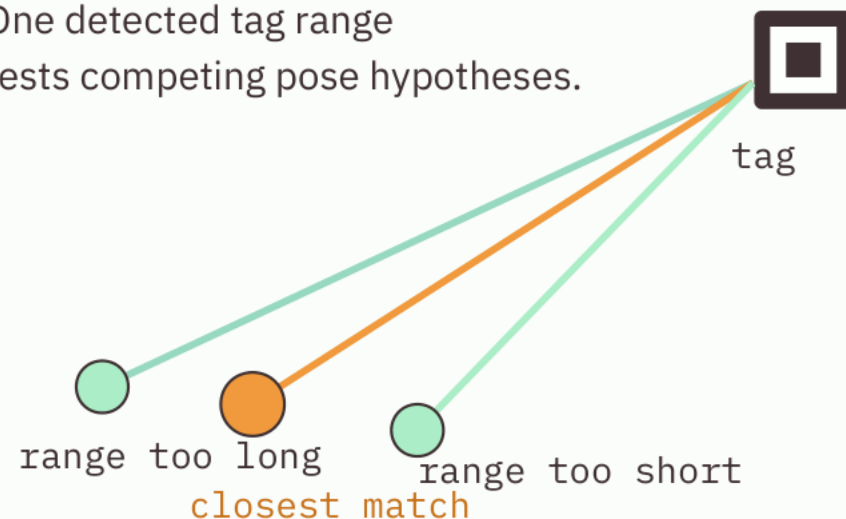
likelihood near
0



particle loses
influence

σ_d controls strictness: smaller uncertainty penalizes range mismatch more sharply; larger uncertainty tolerates more camera and geometry error.

One detected tag range
tests competing pose hypotheses.



Orientation Matters, Not Only Distance

$$L_{\beta,i} = \exp\left(-\frac{1}{2} \left(e_{\beta,i} / \sigma_{\beta} \right)^2 \right)$$

$$L_{\psi,i} = \exp\left(-\frac{1}{2} \left(e_{\psi,i} / \sigma_{\psi} \right)^2 \right)$$

Bearing likelihood

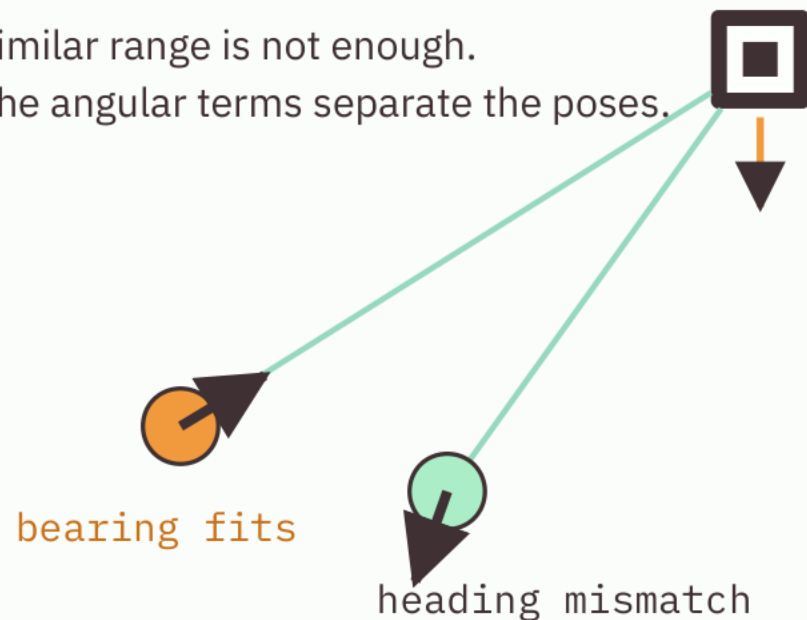
Does the tag appear in the direction predicted by the particle heading?

Yaw likelihood

Is the observed projected tag-face normal consistent with the mapped tag orientation and particle heading?

Smaller angular σ values make heading mismatch lose weight faster. Larger values make the correction more tolerant of pose-estimation noise.

Similar range is not enough.
The angular terms separate the poses.



IMPLEMENTED CORRECTION RULE

Three Independent Checks Become One Weight

distance
match

×

bearing match

×

yaw match

$$q(r, k) = L_d \cdot L_\beta \cdot L_\psi$$

$$w_i = \prod_{\text{real detection } r \text{ in current frame}} \max_{\text{predicted visible tag } k} q(r, k)$$

Each real detection uses the best map-tag prediction available to that particle. The code does not use detected tag identity and does not enforce one-to-one assignment between detections and mapped tags.

IMPORTANT IMPLEMENTATION DETAIL

The previous particle weight is replaced.

The visual update recomputes a fresh frame likelihood for every particle. It does not multiply the new likelihood by the previous stored weight.

$$\bar{w}_i = w_i / \sum_j w_j$$

Normalization turns raw frame scores into the probability distribution used for the estimate, ESS, and resampling.

A

good · good · good
strong

B

good · poor · poor
weak

C

moderate · good · good
medium

Are the Weights Still Healthy?

$$N_{\text{eff}} = 1 / \sum_i \bar{w}_i^2$$

- High ESS: probability is distributed across many useful particles.
- Low ESS: most probability has collapsed onto only a few particles.
- ESS diagnoses degeneracy. It does not modify weights by itself.

ACTUAL GATE IN THE CODE

Resampling is eligible when $N_{\text{eff}} < 0.5 N$

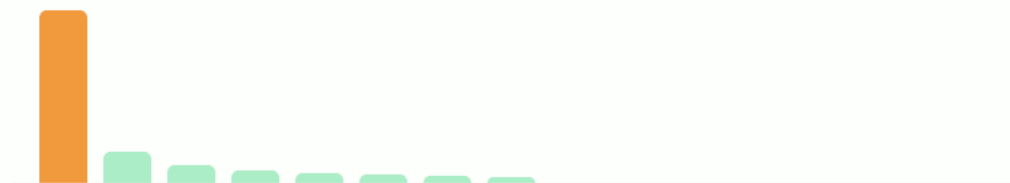
The same branch also requires `frameCount % 4 == 0`.

Healthy weights



Many hypotheses remain relevant → high ESS.

Degenerate weights



A few hypotheses dominate → low ESS.

Duplicate Strong Hypotheses, Remove Weak Ones

```
if tag detections exist:
    recompute normalized frame weights
    ess = 1 / sum(weights ** 2)

    if ess < 0.5 * particle_count \
        and frameCount % 4 == 0:
        particles =
weighted_sample_with_replacement(...)
    add post-resampling noise
    weights = uniform
```

Resampling uses weighted random choice with replacement. Strong particles can be copied several times; weak ones can disappear.

BEFORE

Unefficient population

Many weak particles
+ a few strong particles

AFTER

Focused population

More copies near strong poses
+ fewer poor hypotheses

ESS checked

after each processing loop containing detected tags,
after the frame's observations have been combined

odom-only loop

moves particles only; it does not compute ESS or
resample

multiple tags

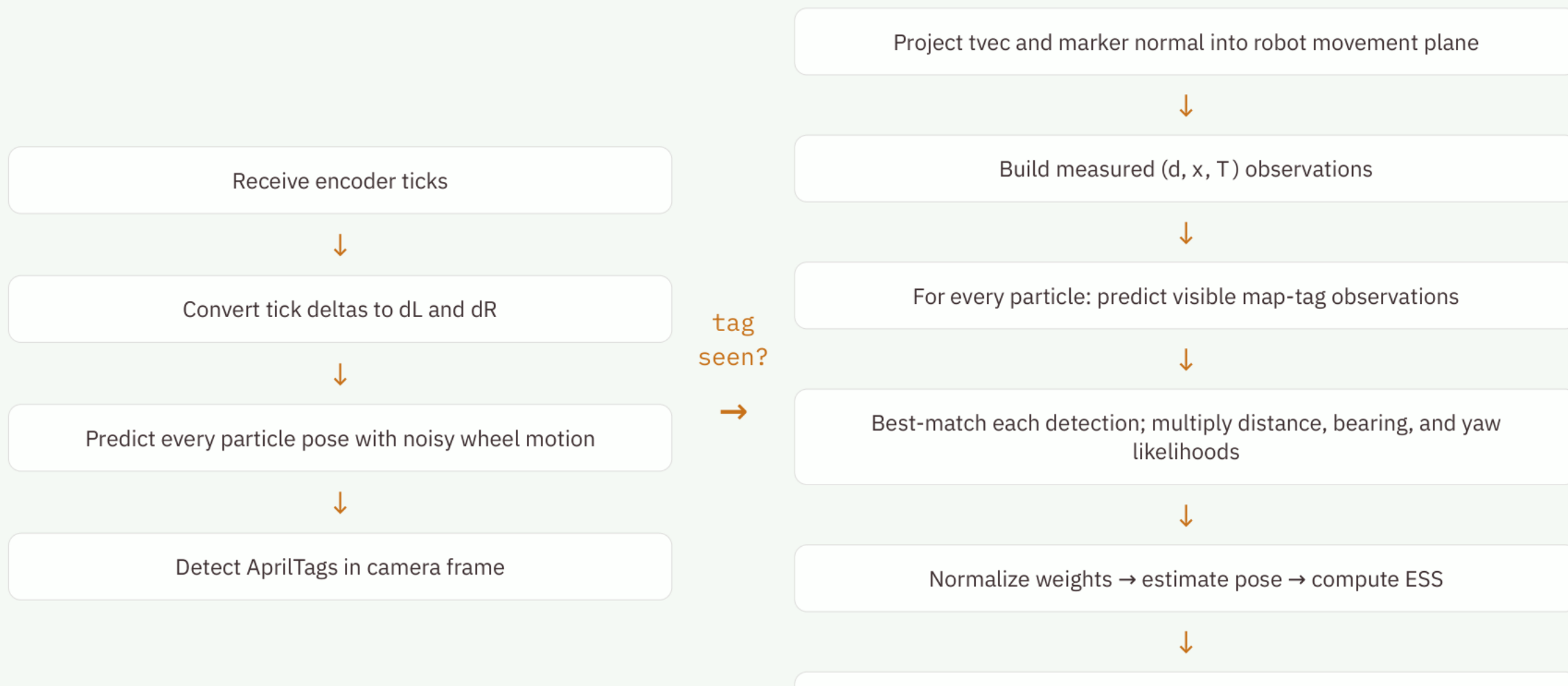
their best-match likelihoods are multiplied into one
particle score before ESS

after sampling

small Gaussian roughening is added to x, y, and D; then
weights reset uniformly

Conditional resampling protects diversity: the code samples only when degeneracy is detected and the frame-count throttle permits it.

Real Duckiebot Localization Loop



KEY TAKEAWAY

The Filter Alternates Between Prediction and Correction

Odometry answers

“How should every hypothesis move?”

Tag observations answer

“Which hypotheses best explain what the camera sees?”

Distance, bearing, and yaw answer

“How convincing is each candidate geometry?”

ESS answers

“Has the weighted population become too concentrated?”

Resampling answers

“Which hypotheses survive, duplicate, and receive fresh local variation?”

Implemented behavior

Fresh visual-frame weights replace stored weights before normalization and adaptive resampling.

Predict → Compare → Weight → Normalize → Check ESS → Resample if needed